

Chapter 35: Module Federation

1. Overview

Module Federation (MF) is a Webpack 5 (and now Rspack/Vite-adjacent) capability that lets independently built and independently deployed JavaScript bundles load code from one another **at runtime**, over the network, as if it were a single application. Before MF, the only ways to share code across separately-deployed frontends were: publish an npm package (build-time coupling, requires a redeploy of every consumer to pick up changes), iframes (isolation but poor UX and no shared state), or Web Components (runtime composition but painful for sharing Angular's DI graph, routing, and change detection across boundaries).

Module Federation solves this by turning each build into a **container** — a small JavaScript runtime that knows how to expose specific modules from itself and how to consume modules exposed by other containers. A **host** application declares which **remote** containers it depends on and which modules it wants from them; a **remote** application declares which of its internal modules it **exposes**. Both host and remote can additionally declare **shared** dependencies (Angular, RxJS, Zone.js, common libraries) so that only one copy of those libraries loads at runtime, avoiding duplicate framework instances.

For Angular specifically, raw Webpack Module Federation config is painful to hand-write because Angular's build process (`@angular-devkit/build-angular`) abstracts Webpack away, and Angular's bootstrapping model has strict requirements around Zone.js being a true singleton. The `@angular-architects/module-federation` library (by Manfred Steyer) exists specifically to bridge that gap: it patches the Angular CLI builder to inject MF configuration, provides a `federation.config.js` convention, and provides runtime helper functions (`loadRemoteModule`) for dynamic loading. Later, the same author's team produced **Native Federation**, which reimplements the Module Federation *runtime concept* using only import maps and standard ES modules — decoupling MF from Webpack entirely so it works with esbuild, Vite, and the modern Angular CLI's default esbuild-based builder.

This chapter goes deep on the mechanics: the plugin's configuration surface, how the runtime resolves and loads a remote at request time, how shared-module version negotiation actually works, and the operational failure modes teams hit in production (singleton violations, version drift, network failures). It assumes you've already read the Micro Frontends chapter covering the broader architectural decision of *whether* to split an app this way; here we assume that decision is made and focus on *how MF makes it work*.

2. Core Concepts

2.1 Host and Remote

- **Host:** the application that is loaded first (typically the shell/container app). It decides, at runtime, to fetch and mount code owned by other teams. A host does not need to expose anything, though it can (a host can also be a remote to another host — "bidirectional" federation).
- **Remote:** an application/build that exposes one or more of its internal modules for consumption. A remote is still a fully working standalone application when run on its own (it has its own `index.html`, router, bootstrap) — MF doesn't change that; it *adds* an additional export surface.

The relationship is not fixed at the code level — it's a build/runtime configuration choice. The same Angular app can be built once and act as a remote to one host and, in a different composition, as a host that loads a different remote.

2.2 The Container and remoteEntry.js

Every remote build emits an additional bootstrap file, conventionally named `remoteEntry.js` (configurable). This file is the **container**: a small runtime bundle that exposes a `get(module)` function (returns a promise resolving to a factory for the requested exposed module) and an `init(shareScope)` function (used to negotiate shared modules — see §4). The container is what a host fetches over the network to gain access to the remote's exposed modules; the rest of the remote's code loads lazily, on demand, as chunks referenced by the container.

2.3 exposes

`exposes` is a remote-side config map from a **public name** to a **local file path**:

```
exposes: {
  './Module': './src/app/remote-entry/entry.module.ts',
}
```

The public name (`./Module`) is what a consumer's `import('remoteName/Module')` resolves against. Each exposed entry becomes its own async chunk, only downloaded when actually requested.

2.4 remotes

`remotes` is a host-side config map from a **local alias** to a **remote container location**:

```
remotes: {
  mfe1: 'mfe1@http://localhost:4201/remoteEntry.js',
}
```

This is the **static/build-time** form — the URL is baked into the host's bundle at build time. Angular Module Federation setups almost always prefer the **dynamic** form instead (see §3.2), where the URL is resolved at runtime from config/environment, precisely so the host doesn't need to know remote URLs at build time and doesn't need to be rebuilt when a remote's deployment URL changes.

2.5 shared

`shared` is the configuration that lets host and remote(s) avoid loading duplicate copies of a library. Key per-package options:

Option	Meaning
<code>singleton: true</code>	Only one version of this package may be active at runtime across host+remotes; if a compatible one is already loaded, reuse it; if versions are incompatible, warn (or error, if <code>strictVersion: true</code>) but still typically fall back to using one version.

Option	Meaning
<code>strictVersion: true</code>	Turns the incompatible-version case from a console warning into a hard runtime error.
<code>requiredVersion</code>	The semver range this build needs; used to compare against what's already in the shared scope. Often set to <code>'auto'</code> (or omitted and inferred from <code>package.json</code> by the CLI helper) so it's read from the installed package version automatically.
<code>eager: true</code>	Bundles the shared module directly into the initial chunk instead of loading it as a separate async request. Needed for the host's entry point in many setups because the very first synchronous code that runs (before any async <code>import()</code> can resolve) can't wait on a network round trip.
<code>includeSecondaries</code>	(helper-library sugar) automatically also shares a package's related sub-entry-points (e.g., sharing <code>@angular/material</code> and its many <code>@angular/material/*</code> entry points) without listing each one.

For Angular apps, the packages that **must** be shared as singletons are `@angular/core`, `@angular/common`, `@angular/router`, `@angular/platform-browser`, `rxjs`, and critically `zone.js`. Anything stateful or identity-sensitive (DI tokens, RxJS operators relying on the same `Observable` prototype, Angular's global module/injector registry, Zone.js's monkey-patched global APIs) breaks if two independent copies exist simultaneously.

2.6 @angular-architects/module-federation

Plain Webpack Module Federation assumes you control `webpack.config.js` directly. The Angular CLI does not expose a Webpack config by default — it uses its own builder abstraction. `@angular-architects/module-federation`:

1. Provides an `ng add` schematic that installs a **custom builder** (`@angular-architects/module-federation:webpack`) which wraps the default Angular builder and merges in a generated Webpack MF config.
2. Introduces the `federation.config.js` file convention (or `webpack.config.js` if you eject further) at the project root, holding `name`, `exposes`, `shared`, and share-mapping options in one place, then internally translates that into the actual `ModuleFederationPlugin` instantiation, applying sensible Angular-specific defaults (auto-sharing based on `package.json`, marking Angular packages singleton by default).
3. Ships runtime helpers — most importantly `loadRemoteModule()` — for **dynamic** remote loading, so a host doesn't need `remotes` entries known at build time at all; it can fetch a manifest (JSON) at runtime mapping remote names to URLs, look up the URL, and load on demand.
4. Provides `initFederation()` / manifest-based bootstrapping to initialize the share scope correctly before Angular's own bootstrap runs (ordering matters — see §4).

2.7 Build-Time vs Runtime Integration

This is the crux tradeoff MF is built to resolve, and interviewers probe it constantly:

	Build-time integration (npm packages / monorepo build)	Runtime integration (Module Federation)
Deploy coupling	Consumer must rebuild+redeploy whenever a shared lib changes	Remote can deploy independently; host picks it up on next page load
CI/CD	Single pipeline typically, or must orchestrate version bumps across repos	Each team owns and ships its own pipeline
Bundle duplication	None (single build's tree-shaking sees everything)	Must be actively managed via <code>shared</code> config; naive setups duplicate frameworks
Version skew	Compile-time type errors catch mismatches early	Mismatches often only surface at runtime, in production, for specific remotes
Rollback	Redeploy previous artifact of the monolith	Roll back just the one remote's deployment
Team autonomy	Low — shared release train	High — independent release cadence per team
Tooling complexity	Simple	Higher — shared scope, manifests, container versioning discipline

2.8 Native Federation

Angular's default build system moved to **esbuild** (via `@angular-devkit/build-angular:application` / the `esbuild`-based builder, and increasingly Vite for dev-server), which does not use Webpack's chunking/runtime model at all — there is no `ModuleFederationPlugin` hook to attach to. Webpack-based Module Federation is therefore fundamentally incompatible with an esbuild/Vite build pipeline.

Native Federation (`@angular-architects/native-federation`) reimplements the same *concept* — exposes/remotes/shared, dynamic runtime loading, singleton negotiation — using only standard web platform primitives: native ES modules and **import maps**, plus a small federation runtime shim (`@softarc/native-federation` / `es-module-shims` for browsers lacking native import-map support). Because it doesn't depend on a Webpack runtime, it works with esbuild, Vite, Rollup, or in principle any bundler that can emit ES modules — which is why the Angular team and community consider it the forward-compatible path as the CLI standardizes on esbuild. Config surface (`federation.config.js` with `exposes` / `shared`) is intentionally kept close to the Webpack-MF helper library's API so migrating between the two is mostly a dependency swap plus adjusted builder name in `angular.json`.

3. Code Examples

3.1 Raw Webpack `ModuleFederationPlugin` — remote

```
// remote-app/webpack.config.js
const { ModuleFederationPlugin } = require('webpack').container;

module.exports = {
```

```

output: {
  uniqueName: 'mfe1',
  publicPath: 'auto',
},
optimization: {
  runtimeChunk: false, // MF manages its own runtime; avoid conflicting webpack runtime
  chunk
},
plugins: [
  new ModuleFederationPlugin({
    name: 'mfe1',
    filename: 'remoteEntry.js',
    exposes: {
      './Module': './src/app/remote-entry/entry.module.ts',
    },
    shared: {
      '@angular/core': { singleton: true, strictVersion: true, requiredVersion: 'auto' },
      '@angular/common': { singleton: true, strictVersion: true, requiredVersion: 'auto' },
      '@angular/router': { singleton: true, strictVersion: true, requiredVersion: 'auto' },
      'rxjs': { singleton: true, strictVersion: true, requiredVersion: 'auto' },
      'zone.js': { singleton: true, strictVersion: false }, // Zone.js has no semver-
        comparable API surface; only one may patch globals
    },
  }),
],
};

```

3.2 Raw Webpack `ModuleFederationPlugin` — host, with static remotes

```

// host-app/webpack.config.js
const { ModuleFederationPlugin } = require('webpack').container;

module.exports = {
  output: { uniqueName: 'shell', publicPath: 'auto' },
  optimization: { runtimeChunk: false },
  plugins: [
    new ModuleFederationPlugin({
      name: 'shell',
      remotes: {
        // static: baked in at build time – requires a host rebuild if the URL changes
        mfe1: 'mfe1@http://localhost:4201/remoteEntry.js',
      },
      shared: {
        '@angular/core': { singleton: true, strictVersion: true, requiredVersion: 'auto',
eager: true },
        '@angular/common': { singleton: true, strictVersion: true, requiredVersion: 'auto',
eager: true },
        '@angular/router': { singleton: true, strictVersion: true, requiredVersion: 'auto',
eager: true },

```

```

    'rxjs': { singleton: true, strictVersion: true, requiredVersion: 'auto', eager: true
  },
    'zone.js': { singleton: true, eager: true },
  },
  }),
],
};

```

`eager: true` on the host's shared entries is necessary because the host's own bootstrap code (which runs synchronously on page load, before any remote is fetched) itself needs `@angular/core/zone.js` immediately — those can't be deferred behind an async container fetch the way a lazily-loaded remote's shared deps can.

3.3 Dynamic remote loading at runtime (no build-time URL)

This is the pattern virtually every real Angular MF setup uses, because it decouples host deploys from remote URLs:

```

// host-app: runtime manifest, resolved from config/environment, not baked into the bundle
// assets/module-federation.manifest.json
{
  "mfe1": "http://localhost:4201/remoteEntry.js",
  "mfe2": "https://cdn.example.com/mfe2/remoteEntry.js"
}

```

```

// host-app/src/app/app.routes.ts
import { Routes } from '@angular/router';
import { loadRemoteModule } from '@angular-architects/module-federation';

export const routes: Routes = [
  {
    path: 'mfe1',
    loadChildren: () =>
      loadRemoteModule({
        type: 'manifest', // resolves 'mfe1' via the loaded manifest, not a hardcoded URL
        remoteName: 'mfe1',
        exposedModule: './Module',
      }).then((m) => m.RemoteEntryModule),
  },
];

```

```

// host-app/src/main.ts – manifest must be loaded and federation initialized BEFORE
bootstrap
import { loadManifest } from '@angular-architects/module-federation';

loadManifest('/assets/module-federation.manifest.json')
  .catch((err) => console.error('Could not load remotes manifest', err))
  .then(() => import('./bootstrap'));

```

```
// host-app/src/bootstrap.ts – the actual Angular bootstrap, split out so it runs AFTER
// the federation runtime + shared scope are initialized
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig).catch((err) => console.error(err));
```

Splitting `main.ts` into a thin loader plus a deferred `bootstrap.ts` is required precisely so the Webpack/Native-Federation runtime can populate the shared scope (see §4) and resolve which shared-module versions win *before* Angular's own module graph starts evaluating — otherwise Angular could initialize using a stale/incorrect shared instance.

3.4 `federation.config.js` (Angular-architects helper library convention)

```
// projects/mfe1/federation.config.js
const { withNativeFederation, shareAll } = require('@angular-architects/native-
federation/config');
// (Webpack variant: const { withModuleFederationPlugin, shareAll } = require('@angular-
architects/module-federation/webpack'))

module.exports = withNativeFederation({
  name: 'mfe1',

  exposes: {
    './Module': './src/app/remote-entry/entry.routes.ts',
  },

  shared: {
    ...shareAll({
      singleton: true,
      strictVersion: true,
      requiredVersion: 'auto',
    }),
  },

  skip: [
    // packages not actually needed at runtime by remotes; trims the shared scope
    'rxjs/ajax',
    'rxjs/fetch',
    'rxjs/testing',
    'rxjs/webSocket',
  ],
});
```

```
// angular.json (relevant excerpt) – swapping the default builder for the federation-aware
one
{
  "projects": {
```

```

"mfe1": {
  "architect": {
    "build": {
      "builder": "@angular-architects/native-federation:build",
      "options": {
        "target": "es2022",
        "outputPath": "dist/mfe1",
        "index": "projects/mfe1/src/index.html",
        "main": "projects/mfe1/src/main.ts",
        "tsConfig": "projects/mfe1/tsconfig.app.json",
        "federationConfig": "projects/mfe1/federation.config.js"
      }
    },
    "serve": {
      "builder": "@angular-architects/native-federation:serve"
    }
  }
}

```

`shareAll()` is helper-library sugar that walks the project's `package.json` and auto-generates `shared` entries for every dependency, so you don't hand-enumerate `@angular/core`, `@angular/common`, `@angular/router`, `rxjs`, etc. one by one — you opt individual noisy sub-paths back out via `skip`.

4. Internal Working

4.1 How a remote container entry is resolved and loaded

1. The host's federation runtime holds a table mapping remote name → container URL (baked in statically, or populated at runtime from a manifest via `loadManifest/loadRemoteModule({ type: 'manifest', ... })`, or via `type: 'module'` pointing straight at a URL).
2. When code requests an exposed module (e.g., the router lazy-loads a route whose `loadChildren` calls `loadRemoteModule(...)`), the runtime checks whether that remote's container script has already been injected as a `<script>` tag. If not, it injects one, pointing at the resolved `remoteEntry.js` URL, and awaits its load.
3. Executing `remoteEntry.js` registers the container object on a well-known global (Webpack MF exposes it as `window[containerScope]`, e.g. `window.mfe1`; Native Federation instead publishes an entry into a browser-native **import map** so the module can be resolved via a normal dynamic `import()` specifier). This is what makes the remote's code addressable without the host needing to have bundled anything from it at build time.
4. The container's `init(shareScope)` is invoked, handing the container a reference to the host's shared-module registry (see §4.2) so the remote can register what it itself would provide, and can pull already-loaded singleton instances instead of loading its own.
5. The runtime calls the container's `get('./Module')`, which returns a promise for a factory function; calling that factory executes (and, if not already fetched, first downloads) the actual exposed module's

chunk(s), returning the real Angular `NgModule` /route/component reference the calling code (e.g. `loadChildren`) expected.

6. Everything downstream — nested lazy chunks the exposed module itself imports — is fetched relative to the remote's own `publicPath` (usually configured as `'auto'` so it's inferred from the URL the container script was actually loaded from, which is what allows the same remote build to be hosted at different URLs across environments without a rebuild).

4.2 Shared-module version negotiation

Each participant (host, and every remote once loaded) maintains a **share scope**: essentially `{ packageName: { version: { get, loaded, eager } } }`. The negotiation algorithm, run whenever a chunk needs a shared module:

1. Look in the local share scope for the package name.
2. If a version already satisfying this consumer's `requiredVersion` range is present **and already loaded** (its factory has already executed), reuse that exact instance — this is the singleton reuse path, and it's what guarantees one `@angular/core` instance across host+remotes.
3. If none is loaded yet but one is *registered* (known but not yet executed) and satisfies the range, that one is chosen and loaded lazily now.
4. If multiple registered candidates satisfy the range, the **highest semver-compatible version** is chosen (Webpack MF's default resolution picks the highest satisfying version among registered providers, not simply "first one seen").
5. If `singleton: true` and the only available version(s) do **not** satisfy the requesting module's `requiredVersion`, the runtime by default emits a console warning and proceeds anyway using the available version ("version might be incompatible" warning), unless `strictVersion: true`, in which case it throws at runtime instead of silently risking incompatible behavior.
6. If no compatible version is registered at all and the module isn't marked shared/singleton, each build simply falls back to bundling and loading **its own private copy** — this is the silent failure mode that produces duplicate framework instances (see §5.1).

The critical mental model: sharing is not "the host always wins" or "whoever loads first wins" — it is a **registry + semver resolution** across whatever has been registered into the shared scope by the time the request is made, which is exactly why *load order* and *eager flags* matter so much in practice, and why misconfigured remotes can non-deterministically end up pulling a different Angular version than the host depending on which chunk happens to request it first.

4.3 Why Angular and Zone.js specifically must be `singleton: true`

- **Zone.js** monkey-patches global async APIs (`setTimeout`, `Promise`, DOM event listeners, XHR) exactly once at load time to build its zone-forking mechanism for change detection. If two independent copies of zone.js load (one bundled into the host, one into a remote that forgot to mark it shared), the second copy re-patches globals a second time, on top of the first patch. This produces duplicate/overlapping monkey-patch layers: change detection can fire twice, in the wrong zone, or async operations can escape Angular's zone entirely, causing UI updates that silently don't render, or that render twice.
- `@angular/core` holds process-global-ish state: the platform injector, `NgModuleRef` registries, and DI's identity-based token resolution. Angular's DI equates tokens by object identity (`InjectionToken` instances, class references). If host and remote each bundle their own copy of `@angular/core`, then

`provideRouter` from one copy and `Router` service instantiated against the *other* copy are, from Angular's own runtime's point of view, two entirely unrelated frameworks running side-by-side in the same page. Cross-app dependency injection (a remote injecting a service the host provided) becomes impossible, `instanceof` checks across the boundary fail, and you frequently get "NG0200: circular dependency" or "no provider for X" errors that look like ordinary DI misconfigurations but are actually a duplicate-instance problem.

- **RxJS** similarly relies on prototype identity for operators and for `instanceof Observable` checks used internally by Angular's `async` pipe and `toSignal`-style interop; a duplicated RxJS can cause `Observable`s created in one copy to fail identity checks performed by code from the other copy.

This is why `@angular-architects/module-federation`'s defaults (via `shareAll`) treat these packages as singleton by default rather than leaving it to each team to remember — it is the single most common Angular-MF misconfiguration, and the helper library exists in large part specifically to prevent it.

5. Edge Cases & Gotchas

5.1 Singleton violations → duplicate Angular instances

Symptoms: `NullInjectorError: No provider for X` for services that *are* provided, `ExpressionChangedAfterItHasBeenCheckedError` cascades that don't correspond to any real template change, change detection appearing to run twice per event, or forms/router state in a remote silently not reflecting host-level state changes. Root cause is almost always one federated participant not marking `@angular/core / zone.js / rxjs` (or a common state library like NgRx) as `singleton: true`, so it bundled and loaded its own private copy instead of joining the shared scope. Fix: audit `shared` config across **every** participant (host and all remotes) — a single remote omitting a shared entry is enough to introduce a second copy, because `shared` is not transitively enforced; each build declares its own list.

5.2 Remote-entry loading failures and the need for network resilience

Unlike a monolith's bundle (all-or-nothing at deploy time), a federated host makes a live network request for `remoteEntry.js` on every navigation to a federated route (subject to browser caching), which introduces a whole new failure class: the remote's server can be down, the CDN URL can 404, a firewall/CORS policy can block the script, or the remote can have deployed a breaking, incompatible container. Production-grade MF hosts must:

- Wrap `LoadRemoteModule` / lazy route factories in explicit error handling and route to a fallback UI ("this section is temporarily unavailable") rather than letting the whole app's router navigation reject.
- Consider timeouts around the container fetch — a hung request otherwise blocks navigation indefinitely.
- Version-pin or at least monitor remote deployments, since a remote team can break the host by shipping a container that no longer exposes the module name the host expects, with zero coordination or host-side deploy in between.

5.3 Version drift between host and remote

Because host and remote are built and deployed independently (that's the entire point), it is easy for their expectations of a shared contract to drift silently: the host's route config expects `./Module` to export `RemoteEntryModule`, but the remote team renamed the exposed symbol; or the remote was built against a newer Angular major version than the host's shared scope tolerates, and `strictVersion` wasn't set, so it runs anyway with a console warning nobody read, degrading in subtle runtime ways instead of failing loudly. Mitigations discussed across the industry: contract tests between host and remote builds in CI, shared TypeScript "shape" packages describing exposed module interfaces, and treating `remoteEntry.js` URLs as versioned artifacts (e.g., `/v2/remoteEntry.js`) rather than always-`latest`, so a host can pin to a known-good remote version and upgrade deliberately.

5.4 Why Native Federation exists — esbuild/Vite incompatibility

Webpack Module Federation is implemented as a Webpack plugin hooking into Webpack's own chunk-graph and runtime-module system — concepts (chunks, runtime modules, `__webpack_require__`) that simply don't exist in esbuild's or Vite's (Rollup-based) build model. Since Angular's CLI has moved its default builder to an esbuild-based pipeline (and the dev server increasingly to Vite) for build-speed reasons, teams building brand-new Angular apps on the modern builder cannot attach `ModuleFederationPlugin` at all — there is no Webpack compilation object for it to hook into. Native Federation was built to give those teams the same exposes/remotes/shared/dynamic-loading capability without requiring a bundler-specific runtime: it relies only on browser-native import maps and standard ESM `import()`, which esbuild, Vite, and Rollup all already emit compatible output for. The tradeoff: Native Federation requires `es-module-shims` (or accepting reduced support) on browsers without native import-map support, and being newer, has a smaller ecosystem/community track record than Webpack MF's several years of production battle-testing.

5.5 Other gotchas worth knowing

- **`publicPath: 'auto'` is mandatory in practice** for remotes — without it, chunk URLs resolve relative to the *host's* origin rather than the remote's own deployed origin, breaking nested lazy chunks the moment host and remote aren't co-located.
 - **CSS/style isolation is not handled by MF at all** — Module Federation only federates JavaScript module graphs; global stylesheet collisions between host and remote (e.g., two different Angular Material theme versions) are a separate problem requiring Shadow DOM, CSS namespacing, or careful shared design-system discipline.
 - **`main.ts` bootstrap ordering**: forgetting to split `main.ts` into an eager bootstrap-loader plus a deferred `bootstrap.ts` (as shown in §3.3) is a very common beginner mistake — Angular's own module evaluation can start racing the federation runtime's shared-scope initialization, causing intermittent (timing-dependent) DI/version errors that are hard to reproduce.
 - **Dev-mode remote/host coupling**: during local development, hosts typically point at `http://localhost:<remote-port>/remoteEntry.js`; forgetting to actually run the remote's dev server before the host is a frequent "why is my route blank" debugging dead-end.
-

6. Interview Questions & Answers

Q1. What problem does Module Federation solve that publishing a shared npm package doesn't?

A: npm packages couple consumers to build time — every consumer must reinstall, rebuild, and redeploy to pick up a change. Module Federation lets independently deployed applications load each other's code **at runtime**, over the network, so a remote team can ship a change and have it appear in the host on the next page load with no host rebuild or redeploy required.

Q2. Define host and remote in Module Federation.

A: A host is the application that consumes modules exposed by other builds at runtime; a remote is a build that exposes some of its internal modules via a container (`remoteEntry.js`). The roles aren't mutually exclusive — a build can be a remote to one host and a host to another remote (bidirectional federation), and a remote is typically still a fully functioning standalone app on its own.

Q3. What is `remoteEntry.js` and what does it actually contain?

A: It's the container bundle a remote emits in addition to its normal application bundle. It's a small runtime script exposing `get(moduleName)` (returns a factory for a requested exposed module, lazily fetching that module's chunk) and `init(shareScope)` (registers/negotiates shared dependencies with whoever loaded it). It does not contain the remote's entire app code — only enough runtime logic to route further requests to the real chunks.

Q4. Why must `zone.js` be shared as a singleton across host and remotes?

Interviewer intent: checks whether the candidate understands Zone.js's mechanism (global monkey-patching), not just that "sharing is good practice."

A: Zone.js patches global async APIs (timers, promises, DOM events, XHR) exactly once to build the zone-forking mechanism Angular's change detection relies on. If a second, independent copy of zone.js loads, it patches those same globals a second time on top of the first, producing overlapping patch layers. This causes symptoms like change detection firing twice, async work escaping Angular's zone so the view doesn't update, or unpredictable double-invocation of zone-aware callbacks. Sharing it as `singleton: true` guarantees exactly one patch layer exists for the whole page.

Q5. Explain the difference between static and dynamic remote configuration.

A: Static: the remote's URL (`mfe1@http://host:port/remoteEntry.js`) is written directly into the host's `remotes` config and baked into the host's build — changing the remote's deployed URL requires rebuilding the host. Dynamic: the host resolves the remote's URL at runtime (from a JSON manifest, environment config, or a backend endpoint) via `loadRemoteModule / loadManifest`, so the actual location can change per environment or over time without any host rebuild. Nearly all production Angular MF setups use dynamic resolution specifically to decouple host and remote deployments.

Q6. What does `singleton: true` do, and what's the difference from `strictVersion: true`?

A: `singleton: true` tells the federation runtime that only one instance of this package may be active across the whole federated application at once — even if the requesting module's own `requiredVersion` doesn't exactly match what's loaded, the runtime will reuse the already-loaded instance rather than loading a second copy, to preserve identity/state consistency. `strictVersion: true` changes what happens when the already-loaded singleton's version doesn't satisfy the requester's semver range: without it, the runtime just logs a console warning and proceeds with the mismatched version; with it, the runtime throws a runtime error instead, converting a silent risk into a loud, catchable failure.

Q7. Walk through what happens, step by step, when a host lazy-loads a federated route.

A: (1) Router triggers `loadChildren`, which calls `loadRemoteModule`. (2) The federation runtime checks if `mfe1`'s container is already injected as a script tag; if not, it resolves the URL (static config or runtime manifest) and injects/loads `remoteEntry.js`. (3) Executing that script registers the container on a global (Webpack MF) or into an import map (Native Federation). (4) The runtime calls the container's `init(shareScope)` to reconcile shared dependencies. (5) It calls `get('./Module')`, awaiting a factory function, then invokes the factory, which fetches (if not cached) and evaluates the actual exposed chunk. (6) The resolved `NgModule`/routes are handed back to the Angular router, which mounts them as if they'd been part of the host's own lazy-loaded bundle.

Q8. How does the runtime decide which version of a shared package "wins" when host and multiple remotes each declare a different version?

Interviewer intent: separates candidates who've only configured `shared: {...}` by copy-pasting docs from those who understand it's a registry + semver resolution, not "host always wins."

A: It is not simply "host wins" or "first loaded wins." Each participant contributes registered candidate versions into a shared per-package registry (the share scope). When a module needs the package, the runtime looks for an already-loaded instance satisfying its `requiredVersion` range first (identity reuse); failing that, among all registered candidates satisfying the range, it picks the highest semver-compatible version and loads that one. If `singleton: true` and no registered version actually satisfies the requester's range, the runtime either warns and proceeds with an available version, or throws, depending on `strictVersion`. Load order matters because a module can only reuse what's already been registered by the time it's evaluated — this is why eager-loading of shared deps in the host's entry point is used to guarantee they're registered before anything else runs.

Q9. Why does `@angular-architects/module-federation` need to hook into a custom Angular CLI builder instead of just working via a normal `webpack.config.js`?

A: The default Angular CLI builders (Webpack-based, historically) don't expose their internal Webpack config for direct editing — the CLI owns and generates it. To inject `ModuleFederationPlugin` configuration, the library ships its own builder (`@angular-architects/module-federation:webpack`) that wraps the standard builder, runs it, and merges in the MF plugin config derived from `federation.config.js`, without requiring the team to eject the CLI's Webpack setup entirely.

Q10. What is the purpose of splitting `main.ts` into a thin bootstrap loader plus a separate `bootstrap.ts`?

A: Module Federation's shared-scope negotiation (deciding which instance of `@angular/core`/`zone.js`/etc. every participant will use) needs to complete *before* Angular's own module graph starts evaluating and instantiating services against those packages. If `main.ts` synchronously imports `@angular/core` and bootstraps immediately, it can race the federation runtime's async initialization (e.g., `loadManifest`'s promise). Splitting into a thin `main.ts` that awaits manifest/federation initialization and only then dynamically `import()`s a `bootstrap.ts` (which does the actual `bootstrapApplication` call) guarantees ordering: shared scope is settled first, Angular bootstraps second.

Q11. What breaks specifically if you don't share `@angular/core` as a singleton between host and remote?

Interviewer intent: tests whether the candidate can connect a config omission to concrete, debuggable symptoms rather than reciting "it causes bugs."

A: Each build ends up with its own private `@angular/core`, meaning its own platform injector, its own `NgModuleRef`/component-factory registries, and its own notion of DI token identity. Symptoms include: services provided by the host being invisible to the remote (`NullInjectorError: No provider for X` even though it's clearly provided somewhere in the page), `instanceof` checks across the host/remote boundary

silently failing, router/state libraries in the remote not reacting to host-driven state changes, and generally two independent "parallel" Angular applications running in the same DOM rather than one coherent app — which is exactly the opposite of what Module Federation was adopted to achieve.

Q12. Why can't you just attach Webpack's `ModuleFederationPlugin` to an esbuild-based or Vite-based Angular build?

A: `ModuleFederationPlugin` is implemented against Webpack-specific internals — its chunk graph, its runtime-module injection mechanism, and its `__webpack_require__` module loader. esbuild and Vite/Rollup have entirely different build and module-loading models with no equivalent hook points, so there is no compilation object for the plugin to attach to. Since the modern Angular CLI defaults to an esbuild-based application builder (and Vite for serving), teams on that builder cannot use Webpack MF at all for new projects; this is precisely the gap Native Federation was created to fill.

Q13. How does Native Federation achieve the same runtime module-sharing behavior without a bundler-specific plugin?

A: It relies on two browser-native (or near-native, via `es-module-shims` polyfill) primitives instead of a bundler runtime: standard ES module `import()` for actually loading code, and **import maps** for the indirection layer that lets a specifier like a remote's exposed module name be resolved to a concrete URL decided at runtime rather than baked in at build time. A small federation runtime (not a Webpack plugin) manages populating/updating the import map and negotiating shared-package versions, mirroring Webpack MF's exposes/remotes/shared semantics but implemented purely with standards-track browser APIs, so it works unchanged whether the underlying bundler is esbuild, Vite, or Rollup.

Q14. In a federated system, what operational safeguards would you put in place around loading remotes, and why does this matter more than in a monolith?

A: Because host and remotes deploy independently, a remote can go down, get slow, or ship an incompatible container at any time with zero coordination with the host's deploy — a failure mode that simply doesn't exist in a single monolithic bundle. Safeguards: wrap `loadRemoteModule` calls with explicit `.catch()` handling and a graceful fallback UI rather than letting router navigation reject outright; add timeouts around the container fetch so a hung remote doesn't block navigation indefinitely; version or otherwise pin remote URLs (rather than always pointing at a floating "latest") so a host can control exactly when it picks up a new remote version; and add contract/integration tests between host and remote builds in CI to catch exposed-module renames or shared-version incompatibilities before they reach production.

7. Quick Revision Cheat Sheet

- **Host** = consumes remote modules at runtime. **Remote** = exposes modules via a `remoteEntry.js` container. A build can be both.
- `exposes` (remote): public name → local file, each becomes its own lazily-fetched chunk.
- `remotes` (host): alias → container URL. Prefer **dynamic** (manifest-based, `loadRemoteModule`) over static so host doesn't rebuild when remote URLs change.
- `shared`: avoid duplicate library instances. Key flags — `singleton` (one instance wins across the app), `strictVersion` (throw vs. warn on incompatible version), `requiredVersion`, `eager` (bundle inline, needed for host's own entry-point deps that run before any async fetch can resolve).
- **Must be singleton for Angular**: `@angular/core`, `@angular/common`, `@angular/router`, `@angular/platform-browser`, `rxjs`, `zone.js` — because Zone.js globally monkey-patches once, and Angular DI/RxJS rely on object identity.

- **@angular-architects/module-federation**: custom CLI builder + `federation.config.js` convention + `loadRemoteModule` / `loadManifest` runtime helpers, because the Angular CLI doesn't expose raw Webpack config.
- **Resolution flow**: router triggers load → runtime resolves/injects `remoteEntry.js` → container `init(shareScope)` negotiates shared deps → `get('./Module')` → factory executes, fetching remaining chunks → Angular router mounts the result.
- **Version negotiation**: registry of candidates per shared package; prefers an already-loaded compatible instance, else picks highest semver-compatible registered candidate; singleton mismatch → warn (default) or throw (`strictVersion`).
- **Build-time vs runtime integration**: build-time (npm/monorepo) = simpler, no duplication, but coupled release trains; runtime (MF) = independent deploys/rollback, but must actively manage shared scope, version drift, and network failure modes.
- **Native Federation** exists because esbuild/Vite have no Webpack-compatible plugin hook; it reimplements `exposes/remotes/shared` using native ES modules + import maps instead of a Webpack runtime, tracking the modern esbuild-based Angular CLI builder.
- **Common failures**: forgotten shared singleton → duplicate Angular/zone.js instances → confusing DI/change-detection bugs; remote down/slow → unhandled navigation rejection; host/remote contract drift → silent warnings or runtime errors depending on `strictVersion`; `main.ts` not split into `loader+bootstrap.ts` → race between federation init and Angular bootstrap.

Created By - Durgesh Singh